

Medical Imaging Visualization Lab Report

Carles Matoses Gimenez¹

¹Universitat Politècnica de Catalunya - BarcelonaTech

Imanol Munoz-Pandiella

Abstract—This report contains information about the real time volumetric rendering shader. It explains how DVR works with the ray marching algorithm and front to back compositing. It also explains the different techniques used to improve the quality of the rendering, such as lighting, transfer functions and optimizations like empty space skipping and early ray termination. Finally, it shows some results and conclusions about the implementation.

Keywords— $\text{\LaTeX}$ class, lab report, academic paper, tau-book class, volumetric rendering.

1. Introduction

In this lab report, we present the implementation of a real-time volume rendering shader using Direct Volume Rendering (DVR) techniques. The shader is designed to visualize 3D volumetric data by employing ray marching and front-to-back compositing methods. Additionally, we incorporate fresnel effect to enhance certain visual aspects of the rendering.

2. Rendering

We are provided a **.raw file containing a row** (or column) of **scalars**. The name of the file contains the dimensions of the volume. We create a **3D texture in WebGL** to sample the volume data. The vertex shader will receive the vertices, normals and texture of a cube as input. For convenience, we will output the vertex position in world space. In the fragment shader, we will receive the 3D texture, light attributes, positions in world space and TF parameters as uniforms.

```
1 vec3 camera;
2 vec3 ray_o = camera;
3 vec3 ray_d = normalize(frag - ray_o);
4 vec3 d = ray_d * step_size(ray_d);
5
6 vec3 p_in, p_out = intersect_box(ray_o, ray_d);
7 p_in += jitter;
8
9 vec3 C = vec3(0.0);
10 float A = 0.0;
11 while rayMarching:
12     vec3 p = p_in + i*d;
13     // Beer-Lambert
14     float alpha = sample_opacity(p, d);
15     if alpha < threshold: continue;
16
17     vec3 normal = -gradient(p, d);
18     vec3 f_phong = phong(p, normal, l, camera, d);
19     float tr = 1.0 - acc_a(p, l, step_size(l-p));
20     f_color = f_phong * tr;
21
22     A += (1.0-A) * alpha;
23     C += (1.0-A) * alpha * f_color;
24
25     if (A >= 0.95) break;
26     i++
27
28 C = pow(C, vec3(1.0/2.2));
29 frag_color = vec4(C, 1.0);
```

Code 1. Pseudocode for DVR

The Direct Volume Rendering could be done in view space to avoid some computations, but **we will do it in world space** for clarity. We start by computing the ray direction from the camera position to the fragment position 1.

We assume volume of size -1 to 1 in all axis, this can be extended in the future to support arbitrary axis-aligned bounding boxes. The problem is that we do not have any way to know the physical dimensions of the object before it was captured. The best guess would be to assume it was sampled with the same step size in all axis, but this is not always true.

$$\vec{r}_i = \vec{c} - \vec{f} \quad (1)$$

2.1. Ray-Box Intersection

The ray-box intersection is **computed using the slab method** `intersect_box(origin, direction)`. For each axis, we compute the minimum and maximum intersection distances $t_a^{\min}$ and $t_a^{\max}$ using the parametric equation of the ray. Then, we compute $t_{1,a}$ and $t_{2,a}$ as the minimum and maximum of $t_a^{\min}$ and $t_a^{\max}$. Finally, we compute the near and far intersection distances t_{near} and t_{far} as the maximum of $t_{1,a}$ and the minimum of $t_{2,a}$ respectively. If t_{near} is greater than t_{far} , there is no intersection.

2.2. Sampling Step

For the sampling step, we use `step_size(vec3 ray_direction, vec3 dimensions)` that calculates the step size based on the segment we are traversing. The volume is not required to have same voxel size in all axis, we dynamically **calculate the step size for each ray individually based on the ray direction and the volume dimensions**. This method is meant to optimize ray marching to the minimum required steps at any view direction. We use the **Nyquist-Shannon [1] theorem to avoid aliasing**, so we set the step size to half of the minimum distance between voxel centers along the ray direction. This is not done in the function for reutilization purposes.

2.3. Transfer Function

The **transfer function** `transfer_function_sigma(float density, float step_size)` maps the density value to opacity. Color will be implemented in the lighting function. The TF **works as a rectangular window function (also Tukey window)**, where values below a minimum density are fully transparent and values above a maximum density are fully opaque. We add two more controllers to create a smooth transition between the minimum and maximum density values. The implementation is **done with conditionals** to detect the different zones and apply the corresponding formula.

2.4. Beer-Lambert law implementation

Step size opacity independence is achieved with Beer-Lambert law [2] $T = e^{-\sigma \cdot d}$ where T is transmittance, σ is

extinction coefficient and d is distance (step size). Sigma is $\sigma = uTFOpacity * (1.0/d)$, the d terms cancel out. The opacity becomes independent of step size and only depends on uTFopacity.

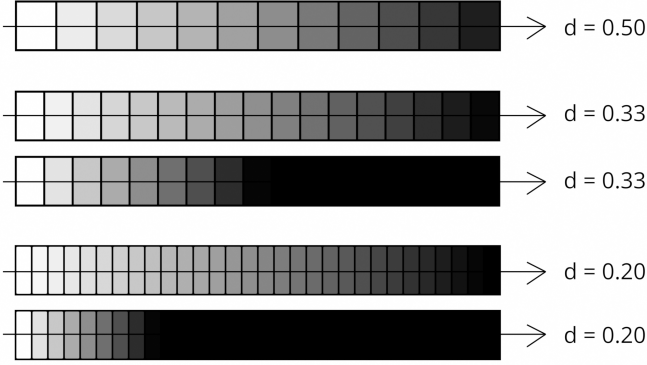


Figure 1. Visualization showing step size independence using Beer-Lambert law normalization. If step size is not taken into account, we darken the volume faster than expected when using smaller step sizes, leading to inconsistent rendering results.

2.5. Light Position

We assume scene has a unique light source. Additionally, light intensity does not diminish with distance for simplicity. The light position is provided as a uniform variable to the fragment shader. For soft shadows (minimum of two light sources), we **generate points in a disc** from which we will calculate the light contribution. The disc is generated in the plane **orthogonal to the light-to-fragment direction**, with a radius controlled by the UI. This method simulates an area light source, producing softer shadows by averaging the contributions from multiple sample points on the disc.

2.6. Color Accumulation

Color accumulation is performed in `accumulate_color_and_opacity(vec3 p_in, vec3 p_out, float step_size)` using front-to-back ray marching. The **ray is sampled uniformly from the near intersection point p_{in} to the far intersection point p_{out}** . At each step, the volume density is converted to an extinction coefficient σ via the transfer function.

Opacity is computed using the Beer-Lambert law over the current step length d :

$$\alpha = 1 - \exp(-\sigma \cdot d) \quad (2)$$

If the sample opacity exceeds a small threshold, **gradients are evaluated to estimate the local normal, and Phong lighting [3] is applied** using `computeLighting`. The lighting model combines ambient, diffuse, and specular terms:

$$\mathbf{C}_{sample} = \mathbf{C}_{ambient} + \mathbf{C}_{diffuse} + \mathbf{C}_{specular} \quad (3)$$

with standard Phong definitions for each component. The shininess parameter α is controlled through the UI (light radius).

To account for light absorption between the current sample position and the light source, the **shaded color is multiplied by a transmittance term computed along the shadow ray using `accumulate_opacity`**. This secondary ray march

calculates the step size based on the light direction. For optimization reasons we do not reduce the step size by half when calculating shadows (becomes unnecessary expensive for non noticeable improvements).

Color and opacity are accumulated using **premultiplied alpha compositing in a front-to-back manner like in [4] and [5]:**

$$\mathbf{C}_{acc} \leftarrow \mathbf{C}_{acc} + (1 - \alpha_{acc}) \cdot (\mathbf{C}_{sample} \cdot \alpha) \quad (4)$$

$$\alpha_{acc} \leftarrow \alpha_{acc} + (1 - \alpha_{acc}) \cdot \alpha \quad (5)$$

This formulation corresponds to a Riemann sum approximation of the volume rendering integral and matches the Beer-Lambert extinction model used in the implementation.

For efficiency, **early ray termination** is applied: once the accumulated opacity exceeds a threshold (e.g. $\alpha_{acc} > 0.95$), the ray marching loop is terminated since further contributions become negligible.

2.7. Gradient Computation

We have tried **two methods to compute the gradients 2**, both with advantages and disadvantages. The **first method uses density values**. This means we do not take the TFs into account. This is perfect for voxels in the margins so they **can show normals even if the TF returns maximum opacity in a rectangular window**. This gives the best results for most cases. **Hollow objects will have inverted normals if looked from the inside**. If TF removes bones and muscles to only leave skin, we will not be able to illuminate it with the current diffuse lightning since normal directions will point outwards as expected. The **second approach is to use opacity values** from the TF. We solve the illumination of hole objects but **we loose density information** when calculating the gradient.

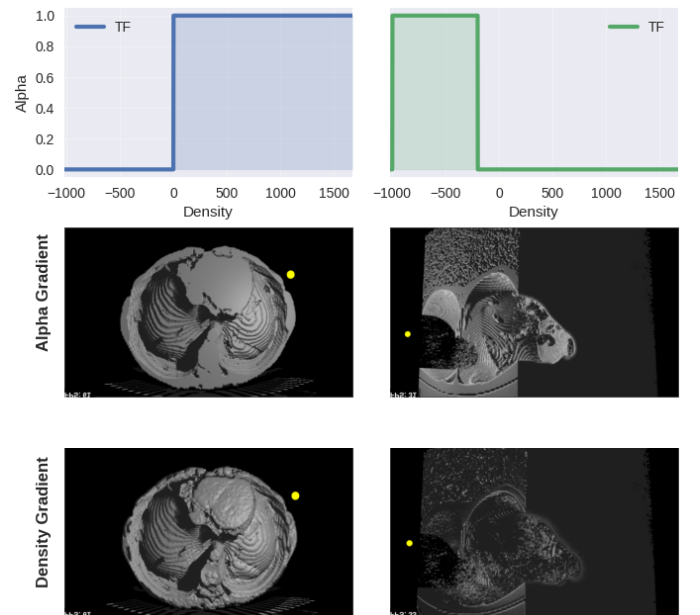


Figure 2. Comparison of normals computed with opacity (left) and density (right). Notice how the opacity normals fail to capture the surface details due to the transfer function removing important density information. Density normals fail to provide correct illumination for hollow objects when viewed from the inside (normals are inverted).

The gradient ∇f at point $\mathbf{p}$ is **computed using central differences**. We sample the volume at six neighboring points displaced by $\Delta s \cdot c_{\text{smooth}}$, where Δs is half the step size and c_{smooth} is a smoothing coefficient:

$$\mathbf{p}_l = \mathbf{p} + (-\Delta s \cdot c_{\text{smooth}}, 0, 0) \quad (6)$$

$$\mathbf{p}_r = \mathbf{p} + (\Delta s \cdot c_{\text{smooth}}, 0, 0) \quad (7)$$

$$\mathbf{p}_b = \mathbf{p} + (0, -\Delta s \cdot c_{\text{smooth}}, 0) \quad (8)$$

$$\mathbf{p}_t = \mathbf{p} + (0, \Delta s \cdot c_{\text{smooth}}, 0) \quad (9)$$

$$\mathbf{p}_{\text{close}} = \mathbf{p} + (0, 0, -\Delta s \cdot c_{\text{smooth}}) \quad (10)$$

$$\mathbf{p}_{\text{far}} = \mathbf{p} + (0, 0, \Delta s \cdot c_{\text{smooth}}) \quad (11)$$

The gradient is then computed as:

$$\nabla f(\mathbf{p}) = \begin{pmatrix} f(\mathbf{p}_r) - f(\mathbf{p}_l) \\ f(\mathbf{p}_t) - f(\mathbf{p}_b) \\ f(\mathbf{p}_{\text{far}}) - f(\mathbf{p}_{\text{close}}) \end{pmatrix} \quad (12)$$

where $f(\mathbf{p})$ represents the sampled density (or opacity) value at position $\mathbf{p}$. The gradient vector points in the direction of maximum density increase. For surface normals, we negate this vector to obtain the outward-pointing normal:

$$\mathbf{n} = -\frac{\nabla f(\mathbf{p})}{|\nabla f(\mathbf{p})|}.$$

Additional problems to normal calculations: **If using density, we need to know the minimum possible value** of density since "void" will take that value (or smaller) for sampling out of the volume points. If done correctly, the model should have values between 0 and infinity but this is not the case in the medical images used in the warmup (from -1024 to 1637).

3. Silhouette

Using the Fresnel effect, we can **highlight the silhouette of dense parts** of the volume. The classical Schlick approximation is

$$F(\theta) = F_0 + (1 - F_0)(1 - \cos \theta)^u \quad (13)$$

where F_0 is the base reflectance at normal incidence, θ is the angle between the view direction and the surface normal and u controls the rim sharpness. In our implementation we drop F_0 (set to 0) and use a simplified rim term driven by the view-normal cosine:

$$\text{extrim} = (1 - |\mathbf{v} \cdot \mathbf{n}|)^p, \quad p = 6.0 \quad (14)$$

To keep the effect only on semi-opaque regions **we gate it with the accumulated opacity using a smoothstep mask**: $w = \text{smoothstep}(0.3, 0.7, \alpha) \text{rim}$. This emphasizes edges (grazing angles) while avoiding over-brightening fully transparent or fully opaque areas 5. Figure 3 shows the effect, and Fig. 6 illustrates the masking window.

4. Performance

4.1. GPU usage and limitations

GPUs achieve high parallelism by organizing threads into groups called warps, with **each warp typically consisting of 32 threads** that execute instructions simultaneously. For

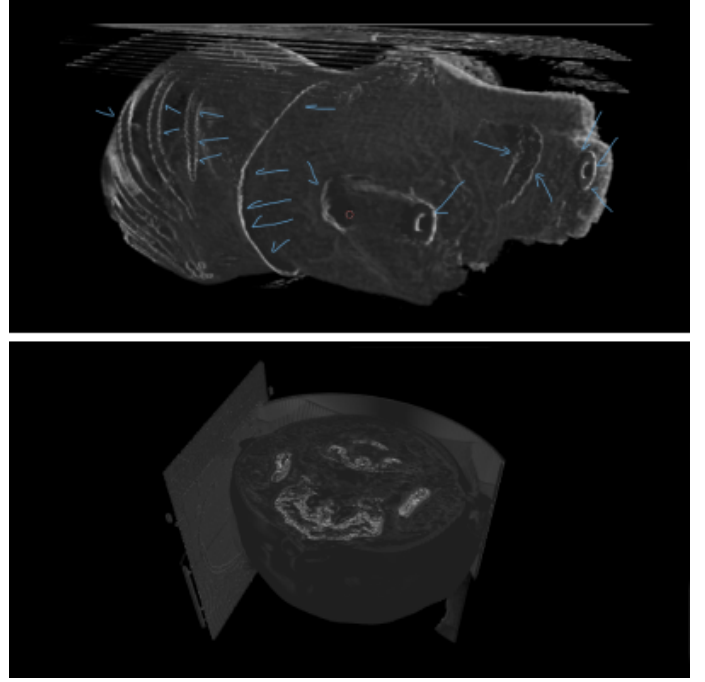


Figure 3. Silhouette highlighting using the Fresnel effect to emphasize dense regions of the volume. Rendered with a single light behind the model.

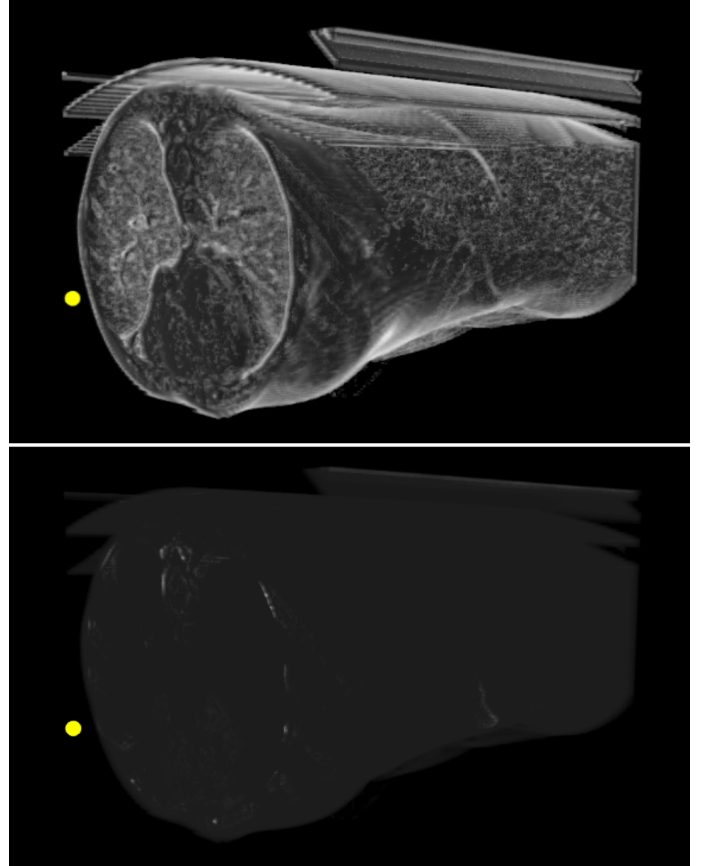


Figure 4. Comparison of volume rendering with and without smoothstep masking of the Fresnel effect. The top image shows excessive brightening in non-dense areas without masking, while the bottom image uses smoothstep to limit the effect to highly dense regions, resulting in a more visually appealing highlight.

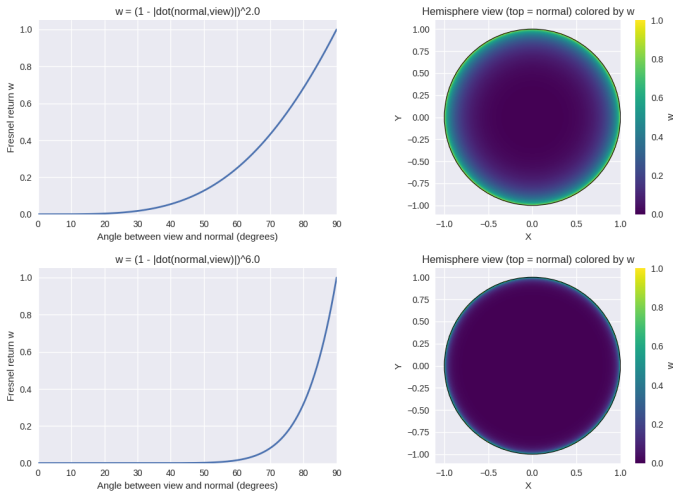


Figure 5. Fresnel function showing the relationship between viewing angle and edge intensity.

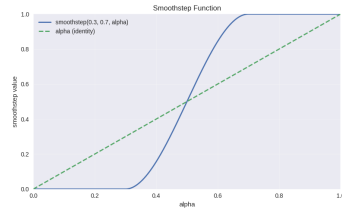


Figure 6. Smoothstep function used to limit the Fresnel effect to specific opacity ranges.

example, an NVIDIA RTX 3080 features 8704 CUDA cores, allowing up to 272 warps to run in parallel ($8704/32 = 272$). When rendering a 1920×1080 image (2,073,600 pixels), **approximately 7617 warps are required to process all fragments** ($2,073,600/272 \approx 7617$). Since each warp may need multiple cycles to complete complex tasks like ray marching, efficient utilization depends on freeing warps quickly. Techniques such as early ray termination and empty space skipping help release warps sooner, enabling the GPU to process more fragments in parallel and avoid underutilization caused by long-running warps.

It is necessary to understand that a **warp will not finish until all its threads have finished**. This means that **if one thread takes a long time to compute, the whole warp will be delayed**. This is why we set a maximum number of steps for the ray marching. Another problem we perceived even though it is not too important for this project is that making a war try to **access memory in non-coalesced way will make it take much longer to fetch the data**. This happened when calculating the gradient with a large smoothing factor, since each thread in the warp was trying to access different parts of the texture memory.

4.2. Early termination

As mentioned before, we implement ... early termination techniques to improve performance:

- **Opacity thresholding:** In the camera ray or the fragment-to-light rays, if the accumulated opacity exceeds a certain threshold (e.g., 0.95), we terminate the ray marching early since further contributions will be negligible. **This is the most effective optimization.**

- **Maximum step limit:** In some cases, we can run into scenarios where rays traverse large empty spaces or low-density regions (mostly if i implemented something wrong). This should prevent crashes but not really optimize performance unless the volume has really high resolution.
- **Empty space skipping:** If the ray is traversing outside the volume bounds, we assume the step has exited the volume and terminate the ray marching. This is usually the case when a ray does not reach maximum opacity before exiting the volume. There is an improvement in performance, mostly when we have multiple lights and each light needs to compute a shadow ray.

Table 1. Performance impact of early termination techniques. Each row disables one optimization to show its effect on rendering speed (measured in frames per second, FPS). Tested on volume of size $128 \times 128 \times 100$ with adaptive step size and 4 light sources. Rendered with a GTX1060 GPU.

Configuration	FPS
All optimizations enabled	44 FPS
Without opacity thresholding	10 FPS
Without maximum step limit	44 FPS
Without empty space skipping	21 FPS

5. Conclusions

While dynamically selecting the sampling step size based on the axis resolution seems theoretically optimal, in practice it introduces noticeable artifacts. Scaling the opacity by the step size should normalize the results, but diagonal rays traversing the volume often use larger steps and can miss important features, leading to inconsistent rendering. Despite the mathematical justification, the normalization does not behave as expected, especially for non-axis-aligned rays. Ultimately, using a fixed sampling rate determined by the minimum voxel size produces artifact-free images, even though it may be slightly less efficient computationally.

References

- [1] H. Nyquist, "Certain topics in telegraph transmission theory," *Transactions of the American Institute of Electrical Engineers*, vol. 47, no. 2, pp. 617–644, 1928.
- [2] T. G. Mayerhöfer, S. Pahlow, and J. Popp, "The bouguer-beer-lambert law: Shining light on the obscure," *ChemPhysChem*, vol. 21, pp. 2029–2046, Sept. 2020. Epub 2020 Aug 26.
- [3] B. T. Phong, "Illumination for computer generated pictures," *Communications of the ACM*, vol. 18, no. 6, pp. 311–317, 1975.
- [4] M. Levoy, "Efficient ray tracing of volume data," *ACM Trans. Graph.*, vol. 9, p. 245–261, July 1990.
- [5] M. Levoy, "Display of surfaces from volume data," *IEEE Computer Graphics and Applications*, vol. 8, no. 3, pp. 29–37, 1988.